

TECHNICAL REFERENCE

KitchMemo Data Architecture

Device-identified, account-free shared fridge design

Document version	1.0
Date	2026-08-29
Status	Baseline for team review
Scope	Inventory, sharing, notifications, achievements, and data flows

Core decisions: one device belongs to one fridge at a time; invite-code joining merges fridges; notification content is shared but read state is per device; achievements belong to the fridge.

1. Purpose and scope

This document turns the team's whiteboard discussion into an implementable data model. It covers device bootstrap, personal and shared fridges, inventory batches, categories, storage recommendations, usage records, notifications, and achievements. Database fields use English snake_case while explanations follow the document language.

1.1 Confirmed product rules

- The app has no login; an app-generated device_id identifies the device.
- One device manages either one personal fridge or one shared fridge at a time.
- Multiple devices join the same fridge_uid through an invite code and jointly view and edit inventory.
- When joining a shared fridge, the joiner's personal-fridge data is merged into the invite target.
- Shared notification events have common content, but each device has independent read state.
- Achievements belong to the fridge_uid and are shared by all current members.
- Automatic recovery after changing devices is out of scope.

2. Logical database diagram

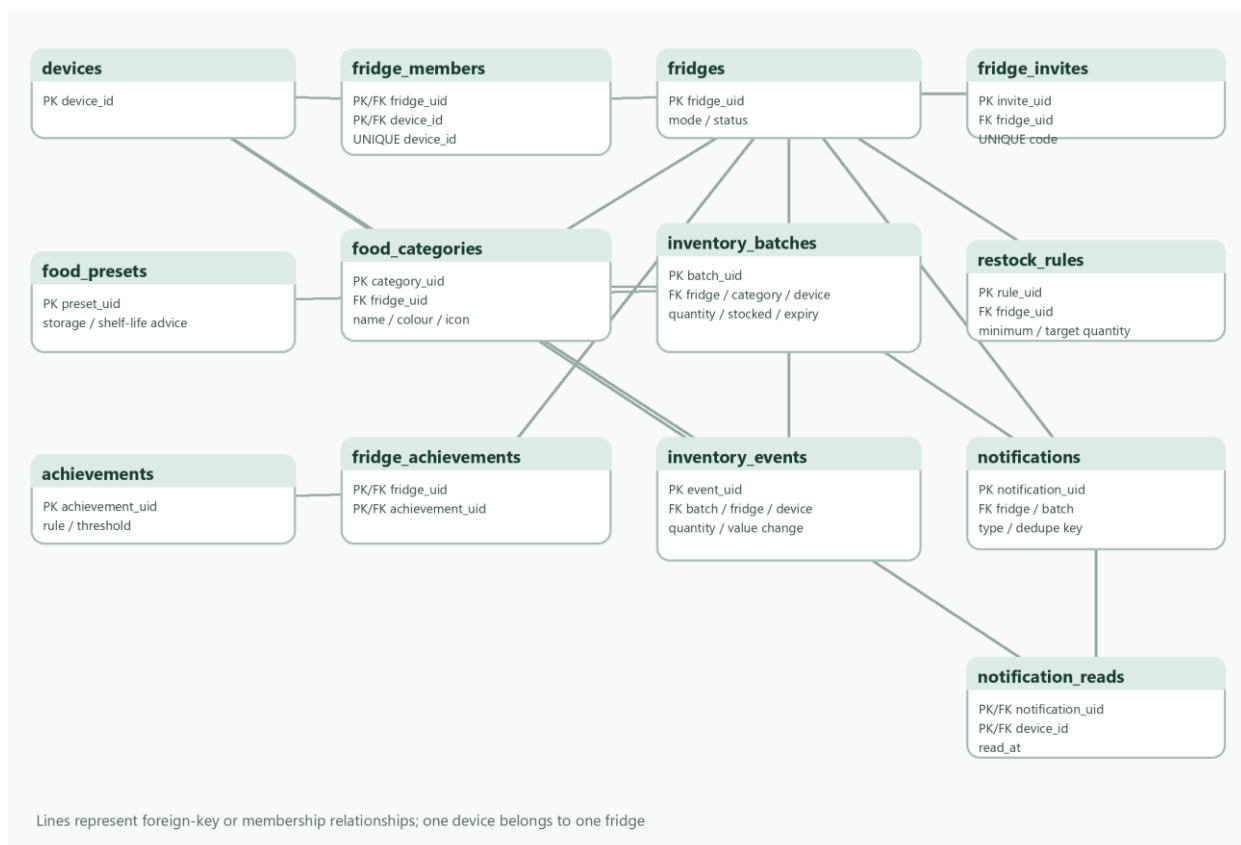


Figure 1: KitchMemo logical entity relationship diagram

3. Relationships and cardinality

Relationship	Cardinality	Meaning
devices → fridge_members	1 : 0..1	device_id is unique, ensuring one active fridge per device.
fridges → fridge_members	1 : N	A personal fridge normally has one member; a shared fridge has several.
fridges → food_categories	1 : N	Default and custom categories belong to one fridge.
fridges → inventory_batches	1 : N	All actual inventory batches are isolated by fridge_uid.
inventory_batches → inventory_events	1 : N	One batch may have many consume, discard, or adjustment events.
notifications → notification_reads	1 : N	Each member device may create its own read record.
fridges ↔ achievements	N : N	fridge_achievements stores unlocked relationships.

4. Data dictionary

The following fields define the recommended logical model. Final Supabase/PostgreSQL SQL can refine enums, indexes, triggers, and deletion policies.

4.1 devices — Device identity

Stores the anonymous identifier generated by the app. It identifies the source of an action but is not an account or trusted authentication mechanism.

Field	Type	Key / constraint	Meaning
device_id	UUID/TEXT	PK	App-generated and persisted device identifier
created_at	TIMESTAMPTZ	NOT NULL	First initialization time
last_seen_at	TIMESTAMPTZ	NULL	Most recent activity time

4.2 fridges — Fridge space

Top-level owner of inventory data. Personal and shared fridges use the same entity and are distinguished by membership and mode.

Field	Type	Key / constraint	Meaning
fridge_uid	UUID	PK	Globally unique fridge-space identifier
name	TEXT	NOT NULL	User-visible name
mode	ENUM	personal/shared	Current personal or shared mode
created_by_device_id	UUID/TEXT	FK devices	Device that created the fridge
status	ENUM	active/merged	Active or merged into another fridge
merged_into_fridge_uid	UUID	FK fridges, NULL	Target fridge after a merge
created_at	TIMESTAMPTZ	NOT NULL	Creation time

4.3 fridge_members — Fridge membership

Links devices to fridges. device_id is unique so one device can belong to only one fridge at a time.

Field	Type	Key / constraint	Meaning
fridge_uid	UUID	PK, FK fridges	Fridge joined by the member
device_id	UUID/TEXT	PK, FK devices, UNIQUE	Member device; unique across the table

Field	Type	Key / constraint	Meaning
joined_at	TIMESTAMPTZ	NOT NULL	Join time

4.4 fridge_invites — Shared-fridge invitations

Creates and validates shared-fridge invite codes while retaining expiry and usage state.

Field	Type	Key / constraint	Meaning
invite_uid	UUID	PK	Invitation identifier
fridge_uid	UUID	FK fridges	Target fridge to join
code	TEXT	UNIQUE, NOT NULL	Invite code entered by the user
created_by_device_id	UUID/TEXT	FK devices	Device that created the invite
expires_at	TIMESTAMPTZ	NULL	Optional expiry time
used_at	TIMESTAMPTZ	NULL	Actual use time
status	ENUM	active/used/revoked/expired	Invite status

4.5 food_categories — Fridge item categories

Stores categories such as vegetables, fruit, meat and eggs, or leftovers. Each fridge owns its category set, so custom categories synchronize across members.

Field	Type	Key / constraint	Meaning
category_uid	UUID	PK	Category identifier
fridge_uid	UUID	FK fridges	Owning fridge
name	TEXT	NOT NULL	Category display name
normalized_name	TEXT	UNIQUE with fridge_uid	Normalized value for deduplication and merging
system_code	TEXT	NULL	Default category code, for example vegetables
colour	TEXT	NULL	Frontend colour token
icon	TEXT	NULL	Frontend icon name
is_default	BOOLEAN	DEFAULT false	Whether initialized by the system
created_by_device_id	UUID/TEXT	FK devices	Device that created the category
created_at	TIMESTAMPZ	NOT NULL	Creation time

4.6 food_presets — Common-food recommendations

Stores global knowledge such as recommending seven chilled days for tomatoes. It only provides defaults; confirmed values are copied to the inventory batch so later preset changes do not alter history.

Field	Type	Key / constraint	Meaning
preset_uid	UUID	PK	Preset identifier
canonical_name	TEXT	UNIQUE, NOT NULL	Canonical food name
aliases	JSONB/TEXT[]	NULL	Search aliases and common spellings
suggested_storage_zone	ENUM	chilled/frozen/pantry	Suggested storage zone
suggested_shelf_life_days	INTEGER	> 0	Suggested shelf-life days
suggested_category_code	TEXT	NULL	Code mapped to a fridge default category
notes	TEXT	NULL	Optional storage guidance

Field	Type	Key / constraint	Meaning
is_enabled	BOOLEAN	DEFAULT true	Whether the recommendation remains active

4.7 inventory_batches — Inventory batches

Stores actual inventory. Every stocking action creates a new batch even when names match, preserving independent stocking and expiry times.

Field	Type	Key / constraint	Meaning
batch_uid	UUID	PK	Inventory batch identifier
fridge_uid	UUID	FK fridges	Owning fridge
category_uid	UUID	FK food_categories	Category used by frontend filtering
created_by_device_id	UUID/TEXT	FK devices	Device that created the batch
preset_uid	UUID	FK food_presets, NULL	Matched recommendation preset, if any
name	TEXT	NOT NULL	User-confirmed item name
storage_zone	ENUM	chilled/frozen/pantry	Actual storage zone
initial_quantity	NUMERIC(12,3)	> 0	Initial stocked quantity
remaining_quantity	NUMERIC(12,3)	>= 0	Current remaining quantity
unit	TEXT	NOT NULL	Count, gram, millilitre, and so on
purchase_price	NUMERIC(12,2)	>= 0, NULL	Purchase price for the batch
currency	CHAR(3)	DEFAULT AUD	Currency code
stocked_at	TIMESTAMPTZ	NOT NULL	Stocking time
expires_at	TIMESTAMPTZ	NULL	User-confirmed expiry time
opened_at	TIMESTAMPTZ	NULL	Optional opened time
lifecycle_state	ENUM	active/consumed/discarded/archived	Batch lifecycle; does not store freshness state
version	INTEGER	>= 1	Optimistic-lock version for shared edits
created_at	TIMESTAMPTZ	NOT NULL	Creation time
updated_at	TIMESTAMPTZ	NOT NULL	Last update time

4.8 inventory_events — Inventory usage and change log

Records stocking, consumption, disposal, and adjustments. actor_device_id identifies the actor only; the record is shared through fridge_uid. Value impact supports waste and saving metrics.

Field	Type	Key / constraint	Meaning
event_uid	UUID	PK	Event identifier
fridge_uid	UUID	FK fridges	Owning fridge
batch_uid	UUID	FK inventory_batches	Affected inventory batch
actor_device_id	UUID/TEXT	FK devices	Device that performed the action
event_type	ENUM	stock/consume/discard/adjust/merge	Action type
quantity_change	NUMERIC(12,3)	SIGNED	Signed quantity delta; consume and discard are negative
value_change	NUMERIC(12,2)	SIGNED	Consumption is positive benefit, waste is negative impact, stocking is normally zero
occurred_at	TIMESTAMPTZ	NOT NULL	Event time
note	TEXT	NULL	Optional note or adjustment reason

4.9 restock_rules — Restock rules

Provides optional thresholds for the Restock filter. Active batches for the same food are summed before comparison with the minimum quantity.

Field	Type	Key / constraint	Meaning
rule_uid	UUID	PK	Rule identifier
fridge_uid	UUID	FK fridges	Owning fridge
preset_uid	UUID	FK food_presets, NULL	Optional standard food
normalized_item_name	TEXT	NOT NULL	Normalized name used without a preset
minimum_quantity	NUMERIC(12,3)	>= 0	Minimum quantity that triggers restocking
target_quantity	NUMERIC(12,3)	> minimum	Desired quantity after restocking
unit	TEXT	NOT NULL	Rule unit
is_enabled	BOOLEAN	DEFAULT true	Whether enabled

4.10 notifications — Notification events

Stores reminder events shared by the entire fridge. Notification content is shared, but read state is stored separately.

Field	Type	Key / constraint	Meaning
notification_uid	UUID	PK	Notification identifier
fridge_uid	UUID	FK fridges	Owning fridge
related_batch_uid	UUID	FK inventory_batches, NULL	Related inventory batch
notification_type	ENUM	expiring/expired/restock/s ystem	Reminder type
message_key	TEXT	NOT NULL	Localization message key
message_payload	JSONB	NULL	Parameters such as item name and days remaining
dedupe_key	TEXT	UNIQUE	Prevents duplicate events for the same rule
created_at	TIMESTAMPTZ	NOT NULL	Creation time
expires_at	TIMESTAMPTZ	NULL	Notification expiry time

4.11 notification_reads — Per-member notification reads

Stores read state per device. When shared member A reads a notification, member B remains unread.

Field	Type	Key / constraint	Meaning
notification_uid	UUID	PK, FK notifications	Notification that was read
device_id	UUID/TEXT	PK, FK devices	Device that read the notification
read_at	TIMESTAMPTZ	NOT NULL	Read time

4.12 achievements — Achievement definitions

Stores system-supported badge and title definitions independent of a specific device or fridge.

Field	Type	Key / constraint	Meaning
achievement_uid	UUID	PK	Achievement identifier
code	TEXT	UNIQUE, NOT NULL	Stable application code

Field	Type	Key / constraint	Meaning
title_key	TEXT	NOT NULL	Localized title key
description_key	TEXT	NOT NULL	Localized description key
rule_type	TEXT	NOT NULL	Achievement rule type
threshold	NUMERIC	NULL	Unlock threshold
is_enabled	BOOLEAN	DEFAULT true	Whether enabled

4.13 fridge_achievements — Unlocked fridge achievements

Achievements belong to the fridge, so all current members see the same unlocked set.

Field	Type	Key / constraint	Meaning
fridge_uid	UUID	PK, FK fridges	Fridge that earned the achievement
achievement_uid	UUID	PK, FK achievements	Unlocked achievement
unlocked_at	TIMESTAMPTZ	NOT NULL	Unlock time
metric_value	NUMERIC	NULL	Metric value at unlock time

5. Derived states and core constraints

5.1 Top-level filter states

Filter	Source	Rule
Chilled / Frozen / Pantry	inventory_batches.storage_zone	Persisted storage method; only chilled, frozen, or pantry.
Expired	expires_at	lifecycle_state is active and expires_at is earlier than now.
Expiring	expires_at	Not expired and within the configured warning window.
Restock	restock_rules + batch aggregate	Sum active batches by food and unit; remaining total is at or below minimum_quantity.

5.2 Required database constraints

- fridge_members.device_id must be UNIQUE to enforce one fridge per device.

- food_categories must be unique on (fridge_uid, normalized_name).
- inventory_batches.remaining_quantity must remain between zero and initial_quantity.
- expires_at may be null; when present it should normally not precede stocked_at unless historical entry is explicitly confirmed.
- Consumption must insert inventory_events and update remaining_quantity in the same transaction.
- Notifications use dedupe_key for event deduplication; read state must not perform this role.
- All timestamps use TIMESTAMPTZ in UTC and are displayed in the user's timezone.

6. Data flows

6.1 First launch

1. The app generates and persists device_id.
2. It creates devices, a personal fridges row, and fridge_members membership.
3. It copies default categories such as meat and eggs, vegetables, fruit, staples, condiments, drinks, and other.

6.2 Add inventory

4. The user enters an item name and the system searches food_presets by canonical_name and aliases.
5. The UI proposes storage, category, and shelf life; the user may override them.
6. Confirmation creates a separate inventory_batches row and a stock inventory_events row.
7. The same food stocked on different dates creates separate batches; the UI may aggregate them but the database does not.

6.3 Consume, discard, or adjust

8. Locate the exact batch_uid and validate version and remaining quantity.
9. Update remaining_quantity, lifecycle_state, and version in a transaction.
10. Insert inventory_events with actor_device_id, quantity_change, and value_change.
11. Consumption has positive benefit, discard or expiry cleanup has negative impact, and stocking normally has zero impact.

6.4 Generate and read notifications

12. A background job creates notifications from active batches, expiry times, and restock rules.
13. All fridge members read the same notification event set.
14. Opening a notification inserts notification_reads only for the current device_id.
15. The notification remains unread for members without a matching read row.

6.5 Unlock achievements

16. Aggregate consumption, waste, and value impact from inventory_events by fridge_uid.
17. Insert fridge_achievements when an achievements rule is satisfied.
18. Every current member of the fridge immediately sees the achievement.

6.6 Merge fridges through an invite code

1. Validate the invite and obtain the target fridge_uid; lock source and target fridges against concurrent changes.
2. Map categories by normalized_name: reuse matching target category_uid values and move distinct categories.
3. Move source inventory_batches, inventory_events, and restock_rules to the target fridge; same-name batches remain separate.
4. Move the joining device's fridge_members relationship to the target and set the target mode to shared.
5. Do not directly migrate old notifications; expire them and regenerate reminders from the merged active inventory.
6. Recalculate fridge achievements from the merged inventory_events history.
7. Mark the source fridges row as merged and set merged_into_fridge_uid; the complete operation must be one transaction.

7. Query, synchronization, and API boundaries

Screen / action	Primary source	Key filter or write
Fridge home	inventory_batches	fridge_uid + lifecycle_state = active; read batches and optionally aggregate by name in the UI.
Category filter	food_categories + inventory_batches	category_uid, combined with storage_zone and derived states.
Expiring / expired	inventory_batches	Calculate from expires_at and current time; do not persist as fixed categories.
Restock	restock_rules + inventory_batches	Aggregate remaining_quantity by name/preset and unit.
Notification inbox	notifications LEFT JOIN notification_reads	Query by fridge_uid and treat a current-device read row as read.
Achievements	fridge_achievements JOIN achievements	Query only by current fridge_uid; device_id is not required.

7.1 Recommended server operations

- bootstrapDevice: initialize the device, personal fridge, membership, and default categories.
- getCurrentFridge: resolve the unique fridge_members row from device_id.
- createInventoryBatch: apply recommendations and create a batch plus stock event.
- applyInventoryEvent: atomically update batch quantity and insert a usage event.
- createInvite / joinSharedFridge: create an invite and merge fridges transactionally.
- listNotifications / markNotificationRead: read shared notifications and write per-device read state.
- evaluateAchievements: aggregate events by fridge_uid and unlock achievements.

8. Security, recovery, and known limitations

Device identity is not login: `device_id` identifies an installation but does not prove a real user's identity. A client-supplied `device_id` can be spoofed.

Minimum recommended protection: After bootstrap, Express should issue a random device token bound to `device_id`. Later requests submit both values and the server validates `fridge_members`.

Service Role risk: Supabase Service Role bypasses RLS, so every normal Express endpoint must explicitly validate device membership.

Device replacement: The current design does not recover identity across devices. A new device receives a new `device_id` and must initialize again or join through an invite.

Concurrent shared edits: `inventory_batches.version` provides optimistic locking; a version mismatch rejects overwrites and requires refresh.

Deletion strategy: Inventory batches and events should be archived or soft-deleted so achievements, waste metrics, and audit history remain intact.

9. Implementation sequence

1. Create devices, fridges, `fridge_members`, and `food_categories`; complete first-launch bootstrap.
2. Create `food_presets` and `inventory_batches`; connect fridge queries and inventory entry.
3. Create `inventory_events`; implement partial consumption, disposal, and adjustments.
4. Create notifications and `notification_reads`; implement independent member read state.
5. Create achievements and `fridge_achievements`; implement fridge-level achievements.
6. Finally implement `fridge_invites` and transactional fridge merging, including concurrency and rollback tests.

10. Decision summary

The recommended structure uses `fridge_uid` as the common owner of business data, `fridge_members` for device membership, and `inventory_batches` for every independent stocking action. Common-food recommendations remain separate from actual stock; expiry and restock states are derived; notification events are shared by fridge while read state is per device; achievements belong to the fridge. The model supports the current account-free release while preserving a migration path to future accounts.